

주식 거래 분석 API

JPA · MyBatis · AOP

축소 배포된 StockTrading2에 필수 API를 복원하고
분석·통계 기능과 Actuator · AOP를 적용한 기록

과제	백엔드 4일차 실습 — 주식 거래 분석 API 개발
제출자	광주 4반 · G122 박영서
제출일	2026년 8월 6일
저장소	<code>Day17 (8_6)/StockTrading2</code>
환경	Java 21 · Spring Boot 3.2.0 · JPA · MyBatis 3.0.3 · H2 · Gradle 8.5
실행	Swagger <code>localhost:8080/swagger-ui.html</code> · Actuator <code>localhost:8080/actuator</code>
완료	<code>필수 API 8종</code> <code>분석 API 8종</code> <code>Actuator</code> <code>AOP 2종</code> <code>MyBatis</code>

목 차

01 개요

요구사항 · 구현 결과 · 변경 파일

02 필수 API 복원

사용자 · 주식 · 거래 · 포트폴리오 8종

03 매매 트랜잭션

잔액 · 포트폴리오 · 거래 내역을 한 단위로

04 분석 API

평가손익 · 자산 · 수익률 · 통계 (JPA + MyBatis)

05 AOP

실행 시간 측정 · 거래 감사 로그 · 트랜잭션 분리

06 Actuator

health · info · metrics 노출

07 검증과 오류 처리

상태 코드 매핑 · 참조 무결성

08 마무리

검증 결과 · 배운 것 · 더 해볼 것

1. 개요

배포받은 `StockTrading2` 는 실습을 위해 `상당 부분이 제거된 상태`였습니다. 분석 관련 클래스는 주석만 남아 있었습니다.

```
// StockAnalysisController는 모든 기능이 제거되었습니다.  
// 교육생 실습을 위해 이 컨트롤러는 삭제되었습니다.
```

여기에 필수 API를 복원하고, 분석·통계 기능과 운영 관점의 기능(Actuator · AOP)을 붙이는 것이 이번 과제입니다.

캡처 화면의 포트에 대하여 — 애플리케이션의 기본 포트는 `8080` 이지만, 촬영 당시 호스트의 8080 포트가 다른 프로세스에 점유되어 있어 `--server.port=8083` 으로 실행했습니다. 그래서 화면의 주소가 `localhost:8083` 으로 표시됩니다. 동작에는 차이가 없습니다.

요구사항과 결과

필수 API

기능	엔드포인트	결과
User 전체 조회	<code>GET /api/users</code>	200
User 삭제	<code>DELETE /api/users/{id}</code>	204
Stock 수정	<code>PUT /api/stocks/{id}</code>	200
Stock 삭제	<code>DELETE /api/stocks/{id}</code>	204
Stock 코드로 조회	<code>GET /api/stocks/code/{code}</code>	200
Transaction 상세 조회	<code>GET /api/transactions/{id}</code>	200 읽기 전용
Transaction 매매 실행	<code>POST /api/transactions/trade</code>	201
Portfolio 특정 주식	<code>GET /api/portfolios/user/{userId}/stock/{stockId}</code>	200

분석 · 고급 기능 (5개 이상 요구 → 8개 구현)

번호	기능	엔드포인트	구현
①	포트폴리오 평가 손익	GET /api/analysis/portfolio/{userId}	JPA
②	거래 내역 상세	GET /api/analysis/transactions/{userId}	JPA
③	특정 주식 거래 내역	GET /api/analysis/transactions/{userId}/stock/{stockId}	JPA
④	총 자산 조회	GET /api/analysis/assets/{userId}	MyBatis
⑤	총 수익률 조회	GET /api/analysis/return-rate/{userId}	MyBatis
⑥	거래 통계 조회	GET /api/analysis/statistics/{userId}	MyBatis
⑦	일별 거래 내역	GET /api/analysis/daily/{userId}	MyBatis
⑧	거래 감사 로그 (AOP 결과 확인용)	GET /api/analysis/audit/{userId}	JPA

Transaction 쪽 "특정 주식 거래 내역"도 별도로 두어 GET /api/transactions/user/{userId}/stock/{stockId} 로 접근할 수 있게 했습니다.

기능 추가

항목	적용 내용
Actuator	health (하위 구성요소 포함) · info · metrics 등 7종 노출
AOP	① 서비스 실행 시간 측정 ② 거래 감사 로그 기록(별도 트랜잭션)

추가.변경한 파일

```
src/main/java/com/skala/stock/
├─ aop/
│   ├── ExecutionTimeAspect.java      ← 신규 패키지
│   │   서비스 실행 시간 측정
│   └─ TradeAuditAspect.java         ← 신규 패키지
│       거래 감사 로그
├─ exception/
│   ├── GlobalExceptionHandler.java
│   ├── ResourceNotFoundException.java (404)
│   ├── DuplicateResourceException.java (409)
│   ├── ResourceInUseException.java (409)
│   └─ BusinessException.java (400)
├─ controller/ StockAnalysisController ← 복원(8개) · 나머지 3개 확장
├─ service/     StockAnalysisService   ← 복원
│               TradeAuditRecorder.java ← 신규(REQUIRES_NEW 저장)
├─ mapper/      StockMapper.java        ← 복원(집계 쿼리 5종)
│               DailyTradeDto.java      ← 신규
└─ dto/         AssetSummaryDto.java     ← 신규
                ReturnRateDto.java      ← 신규
                ErrorResponse.java       ← 신규

src/main/resources/
├─ mapper/StockMapper.xml              ← 복원(집계 SQL)
└─ application.yml                     ← Actuator 설정

build.gradle                           ← actuator · aop 의존성
```


Swagger

Explore

OpenAPI definition API OAS 3.0

api-docs

Servers

<http://localhost:8083> • Generated server url

사용자 관리 사용자 CRUD API

- [GET /api/users](#) 전체 사용자 조회
- [POST /api/users](#) 사용자 생성
- [GET /api/users/{id}](#) 사용자 조회
- [DELETE /api/users/{id}](#) 사용자 삭제

주식 관리 주식 CRUD API

- [GET /api/stocks/{id}](#) 주식 조회 (ID)
- [PUT /api/stocks/{id}](#) 주식 수정
- [DELETE /api/stocks/{id}](#) 주식 삭제
- [GET /api/stocks](#) 전체 주식 조회
- [POST /api/stocks](#) 주식 생성
- [GET /api/stocks/code/{code}](#) 주식 조회 (종목 코드)

거래 관리 주식 매매제도 거래 API

- [POST /api/transactions/trade](#) 주식 매매 실행
- [GET /api/transactions/{id}](#) 거래 상세 조회
- [GET /api/transactions/user/{userId}](#) 사용자 거래 내역 조회
- [GET /api/transactions/user/{userId}/stock/{stockId}](#) 특정 주식 거래 내역 조회

분석 포트폴리오 평가, 자산 수익률, 거래 통계 API

- [GET /api/analysis/transactions/{userId}](#) [분석ID] 거래 내역 상세 조회
- [GET /api/analysis/transactions/{userId}/stock/{stockId}](#) [분석ID] 특정 주식 거래 내역 조회
- [GET /api/analysis/statistics/{userId}](#) [분석ID] 거래 통계 조회
- [GET /api/analysis/return-rate/{userId}](#) [분석ID] 총 수익률 조회
- [GET /api/analysis/portfolio/{userId}](#) [분석ID] 포트폴리오 평가 순위 조회
- [GET /api/analysis/daily/{userId}](#) [분석ID] 일별 거래 내역 조회
- [GET /api/analysis/audit/{userId}](#) [분석ID] 거래 감사 로그 조회
- [GET /api/analysis/assets/{userId}](#) [분석ID] 총 자산 조회

포트폴리오 관리 포트폴리오 조회 API

- [GET /api/portfolios/user/{userId}](#) 사용자 포트폴리오 조회
- [GET /api/portfolios/user/{userId}/stock/{stockId}](#) 특정 주식 포트폴리오 조회

Schemas

- [StockDto](#)
- [UserDto](#)
- [TradeRequestDto](#)
- [TransactionDto](#)
- [PortfolioDto](#)
- [TransactionStatisticsDto](#)
- [ReturnRateDto](#)
- [DailyTradeDto](#)
- [TradeAuditLog](#)
- [AssetSummaryDto](#)

Swagger UI — 사용자 · 주식 · 거래 · 포트폴리오 · 분석 5개 그룹, 총 24개 엔드포인트

2. 필수 API 복원

종목 코드로 조회 · 전체 조회

`id` 는 DB가 부여한 값이라 외부에서 알기 어렵습니다. 종목 코드 `005930` 은 사용자가 아는 값이므로 별도 조회 경로를 두었습니다.

주소 설계 — `/api/stocks/{id}` 가 이미 한 칸짜리 경로를 차지하고 있어 `/api/stocks/{code}` 로 만들면 충돌합니다. `/code/` 를 한 단계 더 두어 두 칸짜리 경로로 분리했습니다.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** `/api/stocks/code/{code}` (종목 조회 (종목 코드))
- Parameters:** A table with one parameter: `code` (required, string, path) with value `005930`.
- Execute:** A blue button to execute the request.
- Responses:** A section showing the request details and the response.
 - Curl:** `curl -X 'GET' \ 'http://localhost:8083/api/stocks/code/005930' \ -H 'accept: */*'`
 - Request URL:** `http://localhost:8083/api/stocks/code/005930`
 - Server response:** A table with **Code** (200) and **Details**.
 - Response body:**

```
{
  "id": 1,
  "code": "005930",
  "name": "삼성전자",
  "currentPrice": 90000,
  "previousPrice": 70000
}
```
 - Response headers:**

```
connection: keep-alive
content-type: application/json
date: Thu, 06 Aug 2026 08:41:46 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```
- Responses (Summary):** A table with **Code** (200), **Description** (OK), and **Links** (No links).
- Media type:** A dropdown menu set to `*/*`.
- Example Value:**

```
{
  "id": 0,
  "code": "string",
  "name": "string",
  "currentPrice": 0,
  "previousPrice": 0
}
```

GET `/api/stocks/code/005930` — 종목 코드로 삼성전자 조회

GET

/api/users 전체 사용자 조회

^

모든 사용자를 조회합니다

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

curl -X 'GET' \n'http://localhost:8083/api/users' \n-H 'accept: */*'

Request URL

http://localhost:8083/api/users

Server response

Code

Details

200

Response body

[
 {
 "id": 1,
 "username": "user1",
 "password": "password1",
 "email": "user1@example.com",
 "balance": 340000
 },
 {
 "id": 2,
 "username": "user2",
 "password": "password2",
 "email": "user2@example.com",
 "balance": 2000000
 },
 {
 "id": 3,
 "username": "user3",
 "password": "password3",
 "email": "user3@example.com",
 "balance": 3000000
 }
]

Response headers

connection: keep-alive
content-type: application/json
date: Thu, 06 Aug 2026 08:41:41 GMT
keep-alive: timeout=60
transfer-encoding: chunked

Responses

Code

Description

Links

200

OK

No links

Media type

/

Controls Accept header.

Example Value | Schema

[
 {
 "id": 0,
 "username": "string",
 "password": "string",
 "email": "string",
 "balance": 0
 }
]

GET /api/users — 전체 사용자 조회

주식 거래 분석 API · 광주 4반 G122 박영서

7

수정 — 자기 코드는 허용, 남의 코드는 거절

종목 코드에는 `unique` 제약이 있습니다. 수정할 때 다른 종목이 쓰는 코드로는 바꿀 수 없지만, 자기가 쓰던 코드를 그대로 보내는 것은 허용해야 합니다. 그래서 "코드가 바뀌었는지"를 먼저 확인합니다.

```
@Transactional
public StockDto updateStock(Long id, StockDto stockDto) {
    Stock stock = findStockOrThrow(id);

    if (!stock.getCode().equals(stockDto.getCode())
        && stockRepository.existsByCode(stockDto.getCode())) {
        throw new DuplicateResourceException("이미 존재하는 종목 코드입니다: " + stockDto.getCode());
    }

    stock.setCode(stockDto.getCode());
    stock.setName(stockDto.getName());
    stock.setCurrentPrice(stockDto.getCurrentPrice());
    stock.setPreviousPrice(stockDto.getPreviousPrice());

    return convertToDto(stockRepository.save(stock));
}
```


PUT

/api/stocks/{id} 주식 수정

ID로 주식 정보를 수정합니다. 다른 종목이 쓰는 코드로는 바꿀 수 없습니다(409)

Parameters

Cancel

Reset

Name	Description
id * required integer(\$int64) (path)	주식 ID

3

Request body required

application/json

```
{
  "code": "035420",
  "name": "NAVER(주정)",
  "currentPrice": 210000,
  "previousPrice": 200000
}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \
'http://localhost:8083/api/stocks/3' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "code": "035420",
  "name": "NAVER(주정)",
  "currentPrice": 210000,
  "previousPrice": 200000
}'
```

Request URL

http://localhost:8083/api/stocks/3

Server response

Code	Details
200	Response body <pre>{ "id": 3, "code": "035420", "name": "NAVER(주정)", "currentPrice": 210000, "previousPrice": 200000 }</pre> Response headers <pre>connection: keep-alive content-type: application/json date: Thu, 06 Aug 2026 08:41:52 GMT keep-alive: timeout=60 transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	OK Media type */* Controls Accept header. Example Value Schema <pre>{ "id": 0, "code": "string", "name": "string", "currentPrice": 0, "previousPrice": 0 }</pre>	No links

PUT /api/stocks/3 — NAVER 종목명·현재가 수정, 200 OK

주식 거래 분석 API · 광주 4반 G122 박영서

9

삭제 — 참조 무결성을 먼저 확인

`portfolios`와 `transactions`가 `user_id` · `stock_id`를 **NOT NULL 외래키**로 참조합니다. 참조가 남은 채 삭제하면 무결성 제약에 걸려 500이 납니다. 사용자 입장에서는 "서버가 고장났다"로 보이지만 실제로는 **요청이 잘못된 것**이므로, 미리 확인해 409와 이유를 돌려줍니다.

```
@Transactional
public void deleteStock(Long id) {
    Stock stock = findStockOrThrow(id);

    if (portfolioRepository.existsByStockId(id)) {
        throw new ResourceInUseException("보유 중인 포트폴리오가 있어 삭제할 수 없습니다: " +
stock.getCode());
    }
    if (transactionRepository.existsByStockId(id)) {
        throw new ResourceInUseException("거래 내역이 있어 삭제할 수 없습니다: " + stock.getCode());
    }

    stockRepository.delete(stock);
}
```

거래 내역은 함께 지우지 않았습니다. 포트폴리오는 현재 상태라 지워도 되지만, 거래 내역은 "언제 무엇을 얼마에 샀다"는 기록입니다. 사용자를 지운다고 함께 없애면 안 된다고 판단해 삭제를 거절합니다.

DELETE

/api/stocks/{id} 주식 삭제

ID로 주식을 삭제합니다. 포트폴리오-거래 내역이 참조 중이면 삭제할 수 없습니다(409)

Parameters

Cancel

Name	Description
id required	주식 ID
integer(\$int64) (path)	1

Execute

Clear

Responses

Curl

curl -X 'DELETE' \n'http://localhost:8083/api/stocks/1' \n-H 'accept: */*'

Request URL

http://localhost:8083/api/stocks/1

Server response

Code	Details
409 <i>Undocumented</i>	Error: response status is 409

Response body

```
{
  "timestamp": "2026-08-06T17:42:12.050033",
  "status": 409,
  "error": "Conflict",
  "message": "보유 중인 포트폴리오가 있어 삭제할 수 없습니다: 005930",
  "path": "/api/stocks/1"
}
```

Download

Response headers

connection: keep-alive
content-type: application/json
date: Thu, 06 Aug 2026 08:42:12 GMT
keep-alive: timeout=60
transfer-encoding: chunked

Responses

Code	Description	Links
200	OK	No links

DELETE /api/stocks/1 — 보유-거래 기록이 있는 종목은 409 Conflict

DELETE /api/users/{id} 사용자 삭제

ID로 사용자를 삭제합니다. 포트폴리오-거래 내역이 있으면 삭제할 수 없습니다(409)

Parameters

Cancel

Name	Description
id required	사용자 ID
integer(\$int64) (path)	1

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
'http://localhost:8083/api/users/1' \
-H 'accept: */*'
```

Request URL

http://localhost:8083/api/users/1

Server response

Code	Details
409 <i>Undocumented</i>	Error: response status is 409 Response body <pre>{ "timestamp": "2026-08-06T17:42:17.077253", "status": 409, "error": "Conflict", "message": "보유 중인 포트폴리오가 있어 삭제할 수 없습니다: user1", "path": "/api/users/1" }</pre> Download
Response headers	
<pre>connection: keep-alive content-type: application/json date: Thu, 06 Aug 2026 08:42:17 GMT keep-alive: timeout=60 transfer-encoding: chunked</pre>	

Responses

Code	Description	Links
200	OK	No links

DELETE /api/users/1 — 포트폴리오를 보유한 사용자도 409

3. 매매 트랜잭션

매매 한 번에 세 가지가 함께 바뀝니다.

1. 사용자 잔액 — 매수면 차감, 매도면 증가
2. 포트폴리오 — 보유 수량과 평균 매수가
3. 거래 내역 기록

중간에 실패하면 셋 다 함께 되돌아가야 하므로 `@Transactional` 로 묶었습니다.

주식 거래 분석 API · 광주 4반 G122 박영서

12

```

@Transactional
public TransactionDto executeTrade(TradeRequestDto request) {
    User user = userRepository.findById(request.getUserId())
        .orElseThrow(() -> ResourceNotFoundException.of("사용자", request.getUserId()));
    Stock stock = stockRepository.findById(request.getStockId())
        .orElseThrow(() -> ResourceNotFoundException.of("주식", request.getStockId()));

    long price = stock.getCurrentPrice();
    long totalAmount = price * request.getQuantity();

    if (request.getType() == Transaction.TransactionType.BUY) {
        buy(user, stock, request.getQuantity(), price, totalAmount);
    } else {
        sell(user, stock, request.getQuantity(), totalAmount);
    }

    Transaction transaction = transactionRepository.save(Transaction.builder()
        .user(user).stock(stock).type(request.getType())
        .quantity(request.getQuantity()).price(price).totalAmount(totalAmount)
        .transactionDate(LocalDateTime.now()).build());

    return convertToDto(transaction);
}

```

평균 매수가 계산

같은 종목을 여러 번 사면 평균 매수를 다시 계산해야 합니다.

```

// 새 평균가 = (기존 평가액 + 이번 매수액) / (기존 수량 + 이번 수량)
long beforeAmount = portfolio.getQuantity() * portfolio.getAveragePrice();
long addedAmount = quantity * price;
long totalQuantity = portfolio.getQuantity() + quantity;

portfolio.setAveragePrice((beforeAmount + addedAmount) / totalQuantity);
portfolio.setQuantity(totalQuantity);

```

매도는 평균가를 바꾸지 않습니다. 파는 것은 이미 산 것의 일부를 넘기는 일이라 "얼마에 샀는지"는 변하지 않기 때문입니다. 수량만 줄이고, 0이 되면 보유 목록에서 제거합니다.

POST

/api/transactions/trade

주식 매매 실행

메수(BUY) 또는 매도(SELL)를 실행합니다. 잔액·보유 수량이 부족하면 400을 반환합니다

Parameters

No parameters

Request body required

application/json

```
{
  "userId": 2,
  "stockId": 3,
  "quantity": 2,
  "type": "BUY"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8083/api/transactions/trade' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "userId": 2,
    "stockId": 3,
    "quantity": 2,
    "type": "BUY"
  }'
```

Request URL

http://localhost:8083/api/transactions/trade

Server response

Code

Details

201

Undocumented

Response body

```
{
  "id": 4,
  "userId": 2,
  "username": "user2",
  "stockId": 3,
  "stockCode": "035420",
  "stockName": "NAVER(네이버)",
  "type": "BUY",
  "quantity": 2,
  "price": 210000,
  "totalAmount": 420000,
  "transactionDate": "2026-08-06T17:41:57.033036",
  "createdAt": "2026-08-06T17:41:57.033151"
}
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Thu, 06 Aug 2026 08:41:57 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code

Description

Links

200

OK

No links

Media type

/

Controls Accept header.

Example Value | Schema

```
{
  "id": 0,
  "userId": 0,
  "username": "string",
  "stockId": 0,
  "stockCode": "string",
  "stockName": "string",
  "type": "BUY",
  "quantity": 0,
  "price": 0,
  "totalAmount": 0,
  "transactionDate": "2026-08-06T08:41:57.0392",
  "createdAt": "2026-08-06T08:41:57.0392"
}
```

POST /api/transactions/trade — NAVER 2주 매수, 201 Created

주식 거래 분석 API · 광주 4반 G122 박영서

14

GET
/api/transactions/{id}
거래 상세 조회

거래 ID로 상세 내역을 조회합니다 (읽기 전용)

Parameters
Cancel

Name	Description
id * required integer(\$int64) (path)	거래 ID 1

Execute
Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8083/api/transactions/1' \
  -H 'accept: */*'

```

Request URL

```
http://localhost:8083/api/transactions/1

```

Server response

Code	Details
200	Response body <pre>{ "id": 1, "userId": 1, "username": "user1", "stockId": 1, "stockCode": "005930", "stockName": "삼성전자", "type": "BUY", "quantity": 10, "price": 70000, "totalAmount": 700000, "transactionDate": "2026-08-06T17:40:39.199151", "createdAt": "2026-08-06T17:40:39.199353" }</pre> Download Response headers <pre>connection: keep-alive content-type: application/json date: Thu, 06 Aug 2026 08:42:02 GMT keep-alive: timeout=60 transfer-encoding: chunked </pre>

Responses

Code	Description	Links
200	OK Media type */* Controls Accept header. Example Value Schema <pre>{ "id": 0, "userId": 0, "username": "string", "stockId": 0, "stockCode": "string", "stockName": "string", "type": "BUY", "quantity": 0, "price": 0, "totalAmount": 0, "transactionDate": "2026-08-06T08:42:02.014Z", "createdAt": "2026-08-06T08:42:02.014Z" }</pre>	No links

GET /api/transactions/1 — 거래 상세 조회(읽기 전용). 사용자·종목 정보가 함께 조회됨

GET
/api/portfolios/user/{userId}/stock/{stockId}
특정 주식 포트폴리오 조회

사용자가 보유한 특정 주식의 수량·평균가·평가손익을 조회합니다. 보유하고 있지 않으면 404를 반환합니다

Parameters
Cancel

Name	Description
userId * required integer(\$int64) (path)	사용자 ID 1
stockId * required integer(\$int64) (path)	주식 ID 1

Execute
Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8083/api/portfolios/user/1/stock/1' \
-H 'accept: */*'

```

Request URL

```
http://localhost:8083/api/portfolios/user/1/stock/1

```

Server response

Code	Details
200	Response body <pre>{ "id": 1, "userId": 1, "username": "user1", "stockId": 1, "stockCode": "005930", "stockName": "삼성전자", "quantity": 6, "averagePrice": 70000, "currentPrice": 90000, "totalValue": 540000, "profitLoss": 120000 }</pre> Download Response headers <pre>connection: keep-alive content-type: application/json date: Thu, 06 Aug 2026 08:42:07 GMT keep-alive: timeout=60 transfer-encoding: chunked </pre>

Responses

Code	Description	Links
200	OK Media type */* Controls Accept header. Example Value Schema <pre>{ "id": 0, "userId": 0, "username": "string", "stockId": 0, "stockCode": "string", "stockName": "string", "quantity": 0, "averagePrice": 0, "currentPrice": 0, "totalValue": 0, "profitLoss": 0 }</pre>	No links

GET /api/portfolios/user/1/stock/1 — 보유 6주 · 평균가 70,000 · 현재가 90,000 · 평가손익 +120,000

4. 분석 API

JPA와 MyBatis를 나눠 쓴 기준

같은 데이터라도 "무엇을 묻느냐"에 따라 도구를 나눴습니다.

도구	쓰는 곳	이유
JPA	엔티티 단위 조회 · CRUD	객체로 다루는 것이 자연스럽고, 연관관계 탐색이 쉽다
MyBatis	GROUP BY · CASE WHEN 집계	SQL을 직접 쓰는 편이 읽기 쉽고 의도가 분명하다

```
-- 종목별 매수/매도 집계: CASE WHEN 으로 한 번의 스캔에서 둘 다 더한다
SELECT
  s.code AS stock_code,
  s.name AS stock_name,
  COALESCE(SUM(CASE WHEN t.type = 'BUY' THEN t.quantity ELSE 0 END), 0) AS total_buy_quantity,
  COALESCE(SUM(CASE WHEN t.type = 'SELL' THEN t.quantity ELSE 0 END), 0) AS
total_sell_quantity,
  COALESCE(SUM(CASE WHEN t.type = 'BUY' THEN t.quantity ELSE -t.quantity END), 0) AS
net_quantity,
  ...
FROM transactions t
JOIN stocks s ON s.id = t.stock_id
WHERE t.user_id = #{userId}
GROUP BY s.code, s.name
```

application.yml 에 map-underscore-to-camel-case: true 가 있어 stock_code → stockCode 로 자동 매핑됩니다. DTO 필드명과 SQL 별칭을 각각의 관례대로 쓸 수 있습니다.

① 포트폴리오 평가 손익

GET

/api/analysis/portfolio/{userId}

[분석①] 포트폴리오 평가 손익 조회

보유 종목별 평가 금액과 평가 손익을 조회합니다

Parameters

Cancel

Name	Description
userId required	사용자 ID
integer(\$int64) (path)	1

ExecuteClear

Responses

Curl

curl -X 'GET' \n'http://localhost:8083/api/analysis/portfolio/1' \n-H 'accept: */*'

Request URL

http://localhost:8083/api/analysis/portfolio/1

Server response

Code	Details
200	Response body [{"id": 1, "userId": 1, "username": "user1", "stockId": 1, "stockCode": "005930", "stockName": "삼성전자", "quantity": 6, "averagePrice": 70000, "currentPrice": 90000, "totalValue": 540000, "profitLoss": 120000}, {"id": 2, "userId": 1, "username": "user1", "stockId": 2, "stockCode": "000660", "stockName": "SK하이닉스", "quantity": 2, "averagePrice": 120000, "currentPrice": 120000, "totalValue": 240000, "profitLoss": 0}]] Response headers connection: keep-alive\ncontent-type: application/json\ndate: Thu, 06 Aug 2026 08:44:14 GMT\nkeep-alive: timeout=60\ntransfer-encoding: chunked

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header.

Example Value | Schema

[{"id": 0, "userId": 0, "username": "string", "stockId": 0, "stockCode": "string", "stockName": "string", "quantity": 0, "averagePrice": 0, "currentPrice": 0, "totalValue": 0, "profitLoss": 0}]]

보유 종목별 평가 금액과 손익 — 삼성전자 6주, 평균가 70,000 → 현재가 90,000

④ 총 자산 · ⑤ 총 수익률

총 자산은 현금 + 보유 주식 평가액입니다. 평가액과 원가 합계는 MyBatis 집계로 구합니다.

```

long holdingValue = nullToZero(stockMapper.selectHoldingValue(userId)); // 수량 × 현재가
long holdingCost = nullToZero(stockMapper.selectHoldingCost(userId)); // 수량 × 평균 매수가
long profit = holdingValue - holdingCost;

// 보유 주식 없으면 원가가 0이라 나눌 수 없으므로 0%로 본다
double rate = (holdingCost == 0) ? 0.0
    : Math.round((profit * 10000.0) / holdingCost) / 100.0; // 소수 둘째 자리

```

GET
/api/analysis/assets/{userId}
[분석@] 총 자산 조회

현금 잔액 + 보유 주식 평가액을 합산한 총 자산을 조회합니다

Parameters
Cancel

Name	Description
userId required	사용자 ID
integer(int64) (path)	1

Execute
Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8083/api/analysis/assets/1' \
-H 'accept: */*'

```

Request URL

```
http://localhost:8083/api/analysis/assets/1

```

Server response

Code	Details
200	Response body <pre>{ "userId": 1, "username": "user1", "cashBalance": 340000, "holdingValue": 780000, "totalAssets": 1120000, "holdingCost": 660000, "evaluationProfit": 120000 }</pre> Download Response headers <pre>connection: keep-alive content-type: application/json date: Thu, 06 Aug 2026 08:44:20 GMT keep-alive: timeout=60 transfer-encoding: chunked </pre>

Responses

Code	Description	Links
200	OK Media type */* Controls Accept header. Example Value Schema <pre>{ "userId": 0, "username": "string", "cashBalance": 0, "holdingValue": 0, "totalAssets": 0, "holdingCost": 0, "evaluationProfit": 0 }</pre>	No links

GET /api/analysis/assets/1 — 현금 340,000 + 평가 780,000 = 총 자산 1,120,000

GET

/api/analysis/return-rate/{userId} [분석⑤] 총 수익률 조회

⌵

평가 손익 ÷ 매입 원가 × 100 으로 수익률을 계산합니다

Parameters

Cancel

Name	Description
userId required	사용자 ID
integer(\$int64) (path)	1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8083/api/analysis/return-rate/1' \
-H 'accept: */*'
```

Request URL

http://localhost:8083/api/analysis/return-rate/1

Server response

Code	Details
200	Response body <pre>{ "userId": 1, "username": "user1", "holdingCost": 660000, "holdingValue": 780000, "evaluationProfit": 120000, "returnRate": 18.18 }</pre> Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Thu, 06 Aug 2026 08:44:25 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header.

Example Value

Schema

```
{
  "userId": 0,
  "username": "string",
  "holdingCost": 0,
  "holdingValue": 0,
  "evaluationProfit": 0,
  "returnRate": 0
}
```

GET /api/analysis/return-rate/1 — 원가 660,000 → 평가 780,000, 수익률 18.18%

⑥ 거래 통계 · ⑦ 일별 거래

GET
/api/analysis/statistics/{userId}
[분석@] 거래 통계 조회

종목별 매수/매도 수량·금액 집계입니다 (MyBatis GROUP BY)

Parameters
Cancel

Name	Description
userId * required integer(\$int64) (path)	사용자 ID

Execute
Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8083/api/analysis/statistics/1' \
  -H 'accept: */*'

```

Request URL

```
http://localhost:8083/api/analysis/statistics/1

```

Server response

Code	Details
200	Response body <pre>[{ "stockCode": "000660", "stockName": "SK하이닉스", "totalBuyQuantity": 2, "totalSellQuantity": 0, "netQuantity": 2, "totalBuyAmount": 240000, "totalSellAmount": 0, "netAmount": -240000 }, { "stockCode": "005930", "stockName": "삼성전자", "totalBuyQuantity": 10, "totalSellQuantity": 4, "netQuantity": 6, "totalBuyAmount": 700000, "totalSellAmount": 280000, "netAmount": -420000 }]</pre> Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Thu, 06 Aug 2026 08:44:30 GMT
keep-alive: timeout=60
transfer-encoding: chunked

```

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header.

Example Value
Schema

```
[
  {
    "stockCode": "String",
    "stockName": "String",
    "totalBuyQuantity": 0,
    "totalSellQuantity": 0,
    "netQuantity": 0,
    "totalBuyAmount": 0,
    "totalSellAmount": 0,
    "netAmount": 0
  }
]
```

종목별 매수·매도 집계 — 순수량(net_quantity)과 순금액(net_amount)까지 한 쿼리로

GET

/api/analysis/daily/{userId} [분석의] 일별 거래 내역 조회

↗

날짜별 거래 건수와 매수·매도 금액 집계입니다 (MyBatis GROUP BY)

Parameters

Cancel

Name	Description
userId required	사용자 ID
integer(\$int64) (path)	1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8083/api/analysis/daily/1' \
-H 'accept: */*'
```

Request URL

http://localhost:8083/api/analysis/daily/1

Server response

Code	Details
200	Response body <pre>[{ "tradeDate": "2026-08-06", "tradeCount": 3, "buyQuantity": 12, "sellQuantity": 4, "buyAmount": 940000, "sellAmount": 280000, "netAmount": -660000 }]</pre> Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Thu, 06 Aug 2026 08:44:36 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK Media type */* Controls Accept header. Example Value Schema <pre>[{ "tradeDate": "string", "tradeCount": 0, "buyQuantity": 0, "sellQuantity": 0, "buyAmount": 0, "sellAmount": 0, "netAmount": 0 }]</pre>	No links

날짜별 거래 건수와 매수·매도 금액 — `FORMATDATETIME` 으로 날짜만 잘라 GROUP BY

③ 특정 주식 거래 내역

GET
/api/analysis/transactions/{userId}/stock/{stockId}
[분석③] 특정 주식 거래 내역 조회

사용자가 특정 종목에 대해 남긴 거래 내역만 조회합니다

Parameters
Cancel

Name	Description
userId required integer(int64) (path)	사용자 ID 1
stockId required integer(int64) (path)	주식 ID 1

Execute
Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8083/api/analysis/transactions/1/stock/1' \
  -H 'accept: */*'

```

Request URL

```
http://localhost:8083/api/analysis/transactions/1/stock/1

```

Server response

Code	Details
200	Response body <pre>[{ "id": 3, "userId": 1, "username": "user1", "stockId": 1, "stockCode": "005930", "stockName": "삼성전자", "type": "SELL", "quantity": 4, "price": 70000, "totalAmount": 280000, "transactionDate": "2026-08-06T17:43:46.975751", "createdAt": "2026-08-06T17:43:46.97579" }, { "id": 1, "userId": 1, "username": "user1", "stockId": 1, "stockCode": "005930", "stockName": "삼성전자", "type": "BUY", "quantity": 10, "price": 70000 }]</pre> Download Response headers <pre>connection: keep-alive content-type: application/json date: Thu, 06 Aug 2026 08:44:46 GMT keep-alive: timeout=60 transfer-encoding: chunked </pre>

Responses

Code	Description	Links
200	OK Media type: */ Controls Accept header. Example Value Schema <pre>[{ "id": 0, "userId": 0, "username": "string", "stockId": 0, "stockCode": "string", "stockName": "string", "type": "BUY", "quantity": 0, "price": 0, "totalAmount": 0, "transactionDate": "2026-08-06T08:44:46.805Z", "createdAt": "2026-08-06T08:44:46.805Z" }]</pre>	No links

/api/analysis/transactions/1/stock/1 — 삼성전자 매수·매도 내역만 필터

5. AOP

두 가지를 적용했습니다. 둘 다 업무 로직과는 무관하지만 여러 곳에 필요한 일입니다.

① 서비스 실행 시간 측정

실행 시간 측정은 모든 서비스 메서드에 필요하지만 업무 로직과는 상관없습니다. 메서드마다 시간 측정 코드를 넣으면 본래 코드가 묻히고, 빠뜨린 메서드가 생깁니다. 한 곳에 모아 두면 서비스 코드는 깨끗해지고 적용 범위도 포인트컷 한 줄로 관리됩니다.

```
@Pointcut("execution(public * com.skala.stock.service..*(..))")
public void serviceMethods() { }

@Around("serviceMethods()")
public Object measure(ProceedingJoinPoint joinPoint) throws Throwable {
    long start = System.nanoTime();
    try {
        return joinPoint.proceed();
    } finally {
        long elapsedMs = (System.nanoTime() - start) / 1_000_000;
        if (elapsedMs >= SLOW_THRESHOLD_MS) {
            log.warn("[AOP] {} 실행 {}ms - 느림", target, elapsedMs);
        } else {
            log.info("[AOP] {} 실행 {}ms", target, elapsedMs);
        }
    }
}
```

`finally` 에 둔 이유는 예외가 나도 시간이 기록되어야 하기 때문입니다. 느린 호출(100ms 이상)은 **WARN** 으로 올려 눈에 띄게 했습니다.

② 거래 감사 로그

"거래를 기록한다"는 요구는 매매 로직 자체와 별개입니다. `executeTrade()` 안에 감사 코드를 넣으면 잔액 계산·포트폴리오 갱신 같은 본래 관심사와 섞입니다. 분리하면 매매 로직은 매매만 다루고, 감사 정책이 바뀌어도 이 클래스만 고치면 됩니다.

```
@AfterReturning(pointcut = "execution(* ...TransactionService.executeTrade(..)) && args(request)",
    returning = "result", argNames = "request,result")
public void logSuccess(TradeRequestDto request, TransactionDto result) { ... }

@AfterThrowing(pointcut = "execution(* ...TransactionService.executeTrade(..)) && args(request)",
    throwing = "ex", argNames = "request,ex")
public void logFailure(TradeRequestDto request, Throwable ex) { ... }
```

여기서 두 번 막혔습니다.

1) 자기 호출 문제 — 처음에는 Aspect 안에 `@Transactional` 메서드를 두고 같은 클래스에서 호출했습니다. 이러면 프록시를 거치지 않아 **REQUIRES_NEW** 가 적용되지 않습니다. 저장 로직을 별도 빈 (`TradeAuditRecorder`)으로 분리해 해결했습니다.

2) 실행 순서 — Aspect가 트랜잭션 어드바이저보다 안쪽에서 돌면 거래 트랜잭션이 아직 살아 있는 상태에서 로그를 남기게 됩니다. `@Order(0)` 을 붙여 트랜잭션보다 바깥에 두었습니다.

왜 별도 트랜잭션(REQUIRES_NEW)인가

실패한 거래(잔액 부족 등)도 "시도했다"는 사실은 남아야 합니다. 거래 트랜잭션에 없으면 롤백될 때 로그까지 함께 사라집니다.

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void record(Long userId, Long stockId, TransactionType type, String message) {
    TradeSnapshotDto snapshot = stockMapper.selectTradeSnapshot(userId);

    auditLogRepository.save(TradeAuditLog.builder()
        .userId(userId).stockId(stockId).type(type).message(cut(message))
        .totalAssets(snapshot.getTotalAssets()) // 그 시점 총 자산
        .totalReturnRate(snapshot.getTotalReturnRate()) // 그 시점 수익률
        .build());
}
```

메시지만으로는 부족하다 — 자산 스냅샷도 함께

처음에는 메시지만 남겼는데, 배포본의 `TradeAuditLog` 엔티티에는 `total_assets` · `total_return_rate` 컬럼이 있고 `TradeSnapshotDto` 도 함께 제공되어 있었습니다. "이 거래 직후 자산이 얼마였는지"를 남기라는 뜻으로 읽혀 스냅샷을 함께 저장하도록 했습니다. 로그만 보고도 그 시점 상태를 알 수 있어야 감사 기록으로서 쓸모가 있기 때문입니다.

```
-- 총 자산·수익률을 한 번의 쿼리로 (MyBatis)
SELECT u.id AS user_id,
       u.balance + COALESCE(hv.holding_value, 0) AS total_assets,
       CASE WHEN COALESCE(hc.holding_cost, 0) = 0 THEN 0
            ELSE ROUND((COALESCE(hv.holding_value, 0) - hc.holding_cost) * 100.0
                      / hc.holding_cost, 2)
       END AS total_return_rate
FROM users u
LEFT JOIN ( ... 보유 평가액 ... ) hv ON hv.user_id = u.id
LEFT JOIN ( ... 매입 원가 ... ) hc ON hc.user_id = u.id
WHERE u.id = #{userId}
```

거래 시점의 수익률이 `0.0%` 인 것은 정상입니다. 방금 산 주식은 **현재가와 평균 매수가가 같아** 아직 손익이 없기 때문입니다. 이후 주가가 오르면 분석 API(`/api/analysis/return-rate`)가 그 변화를 반영합니다.

```

parkyoungseo — parkyoungseo@bag-yeongseoui-MacBookPro — ~ — zsh — 108x32

# 매매 실행 (실패 케이스) - AOP가 감사 로그를 남긴다
{
  "timestamp": "2026-08-06T17:51:29.056377",
  "status": 400,
  "error": "Bad Request",
  "message": "잔액이 부족합니다. 필요 금액: 8999910000, 보유 금액: 3000000",
  "path": "/api/transactions/trade"
}

# 애플리케이션 로그의 [AOP] 항목
[AOP] 거래 실패 - 잔액이 부족합니다. 필요 금액: 4499550000, 보유 금액: 340000
2026-08-06T17:50:45.244+09:00 INFO 30418 --- [skala3-stock-trading-day3] [nio-8083-exec-6] c.skala.stock.aop.ExecutionTimeAspect : [AOP] TradeAuditRecorder.record 실행 0ms
2026-08-06T17:50:45.267+09:00 INFO 30418 --- [skala3-stock-trading-day3] [nio-8083-exec-7] c.skala.stock.aop.ExecutionTimeAspect : [AOP] StockService.updateStock 실행 1ms
2026-08-06T17:50:45.312+09:00 INFO 30418 --- [skala3-stock-trading-day3] [nio-8083-exec-8] c.skala.stock.aop.ExecutionTimeAspect : [AOP] StockAnalysisService.getTotalReturnRate 실행 30ms
2026-08-06T17:50:45.328+09:00 INFO 30418 --- [skala3-stock-trading-day3] [nio-8083-exec-10] c.skala.stock.aop.ExecutionTimeAspect : [AOP] StockAnalysisService.getAuditLogs 실행 2ms
2026-08-06T17:51:29.049+09:00 INFO 30418 --- [skala3-stock-trading-day3] [nio-8083-exec-7] c.skala.stock.aop.ExecutionTimeAspect : [AOP] TransactionService.executeTrade 실행 6ms
[AOP] 거래 실패 - 잔액이 부족합니다. 필요 금액: 8999910000, 보유 금액: 3000000
2026-08-06T17:51:29.055+09:00 INFO 30418 --- [skala3-stock-trading-day3] [nio-8083-exec-7] c.skala.stock.aop.ExecutionTimeAspect : [AOP] TradeAuditRecorder.record 실행 4ms

# 감사 로그 조회 (user1) - 실패 거래도 남아 있다
#4 [BUY] 거래 실패 - 잔액이 부족합니다. 필요 금액: 4499550000, 보유 금액: 340000
#3 [SELL] 거래 성공 - SELL 005930 4주 / 체결가 70000 / 총액 280000
#2 [BUY] 거래 성공 - BUY 000660 2주 / 체결가 120000 / 총액 240000
#1 [BUY] 거래 성공 - BUY 005930 10주 / 체결가 70000 / 총액 700000

```

실패한 매매(400)에도 [AOP] 로그가 남고, 감사 로그에 실패 기록 #4가 보존됨

GET /api/analysis/audit/{userId} [분석@] 거래 감사 로그 조회

AOP(TradeAuditAspect)가 매메 성공/실패 시마다 남긴 감사 로그입니다. 실패한 거래도 별도 트랜잭션으로 기록되어 함께 조회됩니다

Parameters
Cancel

Name	Description
userId * required	사용자 ID
integer(\$int64) (path)	1

Execute
Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8083/api/analysis/audit/1' \
-H 'accept: */*'

```

Request URL

```
http://localhost:8083/api/analysis/audit/1

```

Server response

Code	Details
200	Response body <pre>[{ "id": 4, "userId": 1, "stockId": 5, "type": "BUY", "message": "거래 실패 - 잔액이 부족합니다. 필요 금액: 4499550000, 보유 금액: 340000", "totalAssets": 1000000, "totalReturnRate": 0, "createdAt": "2026-08-06T18:05:26.633575" }, { "id": 3, "userId": 1, "stockId": 1, "type": "SELL", "message": "거래 성공 - SELL 005930 4주 / 체결가 70000 / 총액 280000", "totalAssets": 1000000, "totalReturnRate": 0, "createdAt": "2026-08-06T18:05:26.622602" }, { "id": 2, "userId": 1, "stockId": 2 }]</pre> Download Response headers <pre>connection: keep-alive content-type: application/json date: Thu, 06 Aug 2026 09:06:07 GMT keep-alive: timeout=60 transfer-encoding: chunked </pre>

Responses

Code	Description	Links
200	OK Media type */* Controls Accept header. Example Value Schema <pre>[{ "id": 0, "userId": 0, "stockId": 0, "type": "BUY", "message": "string", "totalAssets": 0, "totalReturnRate": 0, "createdAt": "2026-08-06T09:06:07.4722" }]</pre>	No links

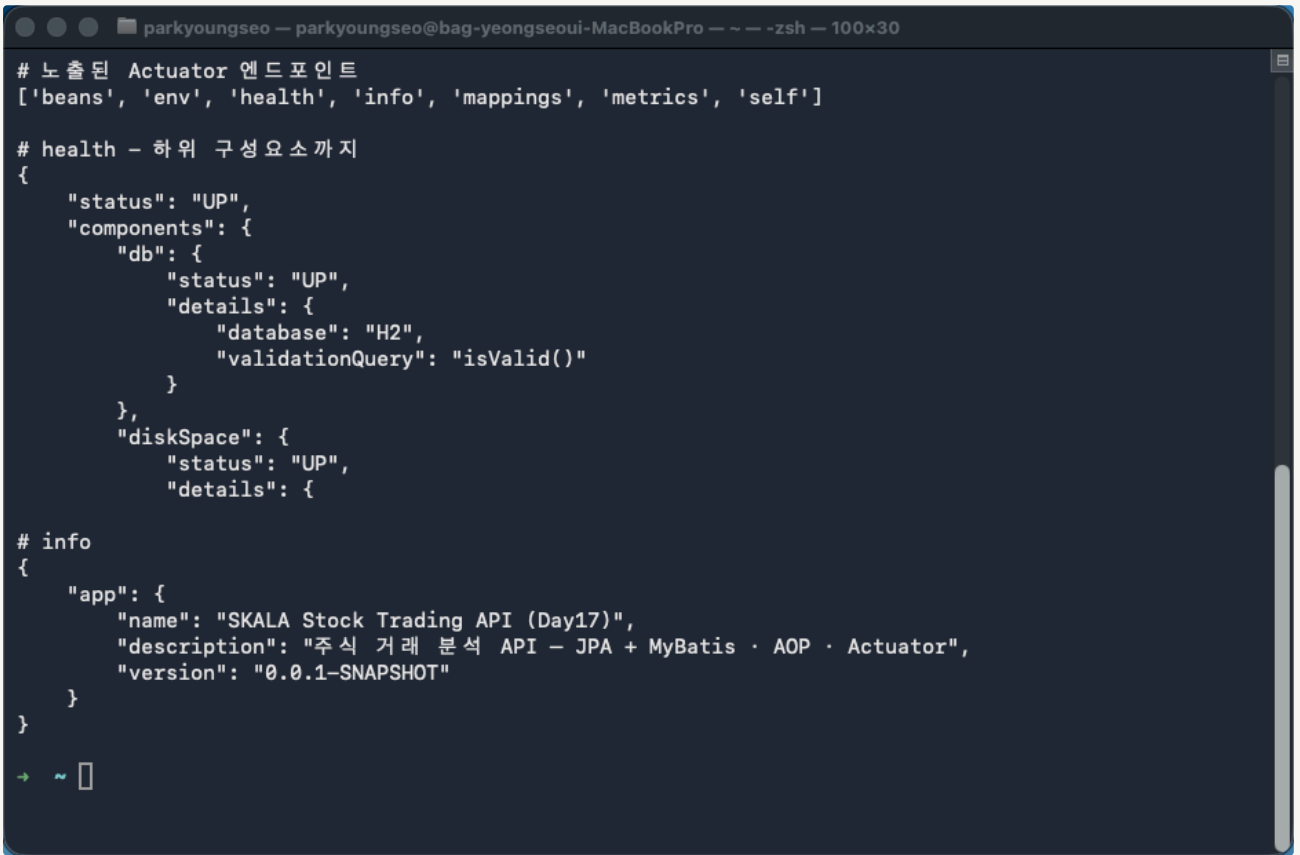
GET /api/analysis/audit/1 — 성공 3건과 실패 1건이 함께 조회됨

6. Actuator

의존성을 추가하고 필요한 엔드포인트를 열었습니다. 기본값은 `health` 하나만 노출됩니다.

```
# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health, info, metrics, env, beans, mappings
  endpoint:
    health:
      show-details: always # 디스크·DB 등 하위 구성요소 상태까지 표시

info:
  app:
    name: SKALA Stock Trading API (Day17)
    description: 주식 거래 분석 API - JPA + MyBatis · AOP · Actuator
```



```
# 노출된 Actuator 엔드포인트
['beans', 'env', 'health', 'info', 'mappings', 'metrics', 'self']

# health - 하위 구성요소까지
{
  "status": "UP",
  "components": {
    "db": {
      "status": "UP",
      "details": {
        "database": "H2",
        "validationQuery": "isValid()"
      }
    },
    "diskSpace": {
      "status": "UP",
      "details": {

```

노출된 엔드포인트 7종 · **health** 하위 구성요소(db · diskSpace · ping) · **info**

/actuator/mappings 로는 실제 등록된 URL 매핑을 확인할 수 있어, 이번처럼 엔드포인트를 대량으로 추가했을 때 누락 여부를 점검하기 좋았습니다.

7. 검증과 오류 처리

배포본에는 전역 예외 처리기가 없어 서비스가 던진 예외가 전부 **500**으로 나갔습니다. 예외 타입을 나누고 **@RestControllerAdvice** 에서 상태 코드로 매핑했습니다.

상황	예외	상태 코드
입력값 검증 실패	<code>MethodArgumentNotValid</code>	400
경로 변수 제약 위반	<code>ConstraintViolation</code>	400
타입 불일치 · 깨진 JSON	<code>TypeMismatch</code> 등	400
잔액 · 수량 부족	<code>BusinessRuleException</code>	400
자원 없음	<code>ResourceNotFoundException</code>	404
없는 경로 · 미지원 메서드	<code>NoResourceFound</code> 등	404 / 405
중복 · 참조 중 삭제	<code>Duplicate</code> · <code>ResourceInUse</code>	409

경로 변수 제약은 클래스에 `@Validated` 가 있어야 동작합니다. 컨트롤러 5곳 모두에 적용해 앱 전체에서 같은 방식으로 걸러지게 했습니다.

```

# 상태 코드 매핑 확인
/api/analysis/assets/0           @Min 위반 → 400
/api/analysis/assets/999       없는 사용자 → 404
/api/stocks/code/999999       없는 종목코드 → 404
/api/portfolios/user/1/stock/3 미보유 → 404
/api/stocks/abc               타입 불일치 → 400
/api/nope                     없는 경로 → 404
DELETE /api/users/1           참조 중 → 409
DELETE /api/stocks           미지원 메서드 → 405

# 검증 실패 응답 본문
{
  "timestamp": "2026-08-06T17:52:07.801583",
  "status": 400,
  "error": "Bad Request",
  "message": "입력값이 올바르지 않습니다",
  "path": "/api/analysis/assets/0",
  "fieldErrors": {
    "userId": "사용자 ID는 1 이상이어야 합니다"
  }
}

# 참조 중 삭제 거절
보유 중인 포트폴리오가 있어 삭제할 수 없습니다: user1
→ ~

```

상태 코드 매핑 8종 확인 · 검증 실패 시 `fieldErrors` 에 필드별 사유 · 참조 중 삭제 거절

8. 마무리

검증 결과

검증 항목	기대	결과
필수 API 8종 (조회 · 수정 · 삭제 · 매매)	200/201/204	통과
분석 API 8종	200	통과
참조 중인 주식 · 사용자 삭제	409	409
잔액 초과 매수	400	400
경로 변수 @Min 위반 · 타입 불일치	400	400
없는 사용자 · 종목 · 미보유 주식	404	404
없는 경로 · 미지원 메서드	404 / 405	404 / 405
총 자산 · 수익률 계산 정확도	수식 일치	일치
AOP 실행 시간 로그	전 서비스	기록됨
AOP 감사 로그 (실패 거래 보존)	롤백 무관	보존됨
/actuator/health	200 UP	UP

계산 검증 — 수치가 맞는지 직접 확인

초기 잔액 1,000,000

BUY 삼성전자 10주 @70,000 -700,000 → 300,000

BUY SK하이닉스 2주 @120,000 -240,000 → 60,000

SELL 삼성전자 4주 @70,000 +280,000 → 340,000 ← 현금 잔액

보유: 삼성전자 6주 @70,000 = 420,000 (현재가 90,000 → 540,000)

SK하이닉스 2주 @120,000 = 240,000 (현재가 120,000 → 240,000)

매입 원가 660,000 / 평가액 780,000 → 평가 손익 +120,000

총 자산 = 340,000 + 780,000 = 1,120,000 ✓ API 응답과 일치

수익률 = 120,000 / 660,000 × 100 = 18.18% ✓ API 응답과 일치

배운 것

- AOP는 "어디에 붙이느냐"만큼 "언제 도느냐"가 중요하다 — 트랜잭션 어드바이저와의 순서에 따라 같은 코드가 다르게 동작한다. @Order 로 명시하지 않으면 의도와 다른 시점에 실행될 수 있다.
- 자기 호출은 프록시를 타지 않는다 — 같은 클래스 안에서 @Transactional 메서드를 불러도 전파 속성이 적용되지 않는다. 별도 빈으로 분리해야 한다.
- 실패도 기록되어야 하는 데이터가 있다 — 감사 로그를 거래 트랜잭션에 얹으면 롤백과 함께 사라진다. REQUIRES_NEW 로 분리하는 이유가 여기 있다.
- JPA와 MyBatis는 경쟁 관계가 아니다 — 엔티티 조작은 JPA, 집계는 SQL이 읽기 쉽다. 한 프로젝트에서 목적에 따라 나눠 쓸 수 있다.
- 매도는 평균 매수가를 바꾸지 않는다 — 도메인 규칙을 코드로 옮길 때 "무엇이 변하고 무엇이 변하지 않는지"를 먼저 정리해야 한다.

더 해볼 것

- `UserDto` 가 응답에 비밀번호를 그대로 포함한다. 요청용·응답용 DTO를 분리하거나 `@JsonProperty(access = WRITE_ONLY)` 가 필요하다.
- 자동 테스트가 없다. 특히 평균 매수가 계산과 트랜잭션 롤백은 MockMvc·단위 테스트로 검증할 가치가 크다.
- 거래 수량·금액을 `Long` 으로 다루고 있어 소수점 단위 가격에는 맞지 않는다. 금액은 `BigDecimal` 이 안전하다.
- 동시에 같은 종목을 매매하면 포트폴리오 갱신에 경합이 생길 수 있다. 낙관적 락(`@Version`)을 고려할 수 있다.
- H2 인메모리라 재시작하면 데이터가 사라진다. 실제 DB와 마이그레이션 도구를 붙이면 스키마를 버전 관리할 수 있다.